

NPTEL Course on
"Computer System Optimization for ML"

Lecture #22

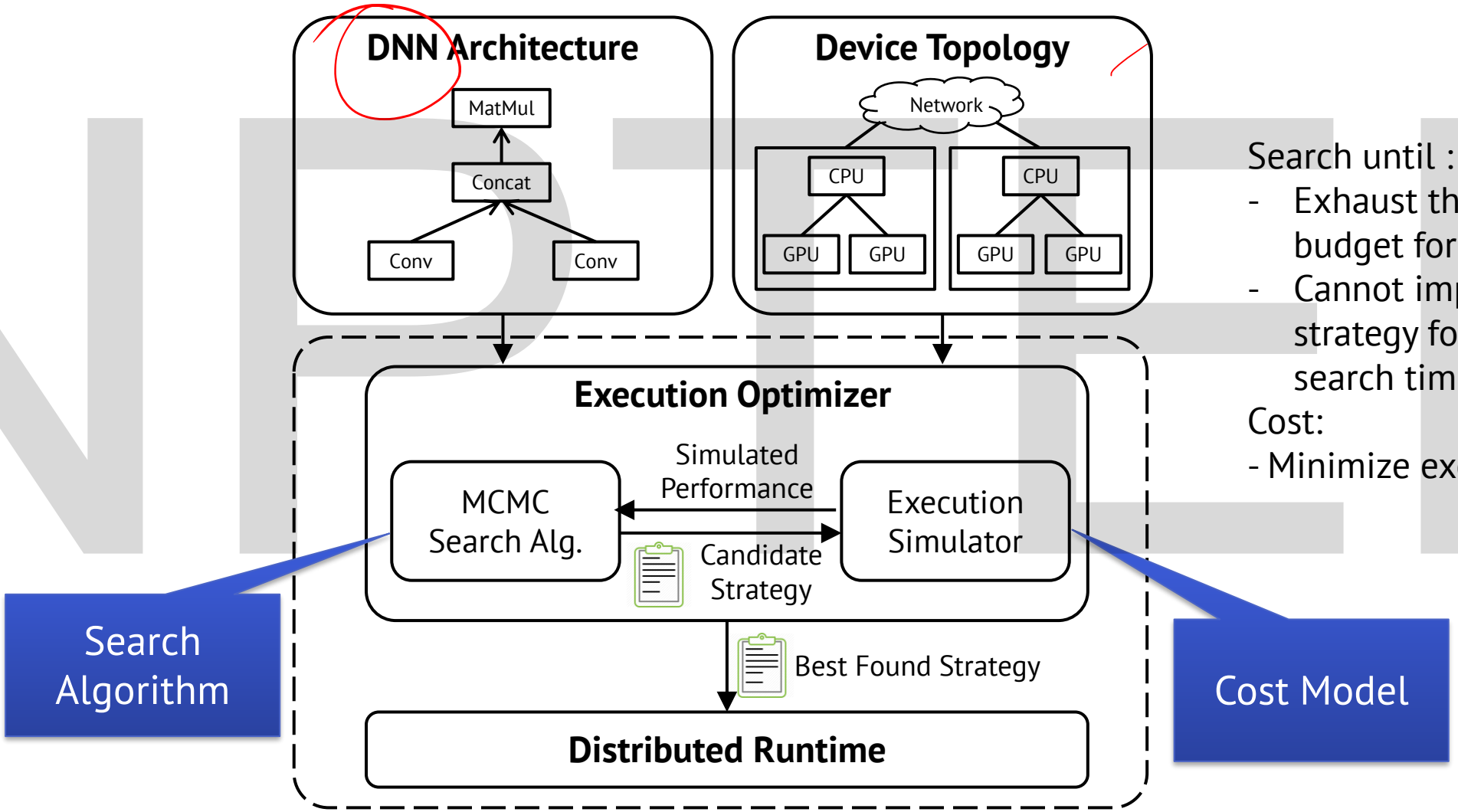
NPTEL

Agenda

- Recap
- Automatic Parallelization (contd.)

NPTEL

FlexFlow



Search until :
- Exhaust the time budget for search (OR)
- Cannot improve the strategy for half the search time

Cost:
- Minimize execution time

Automatic Parallelization Methods

Search-based methods

- MCMC:
  - [Jia et al., 2018]
  - [Jia et al., 2019]
- Heuristics
  - [Fan et al., 2021]

Learning-based methods

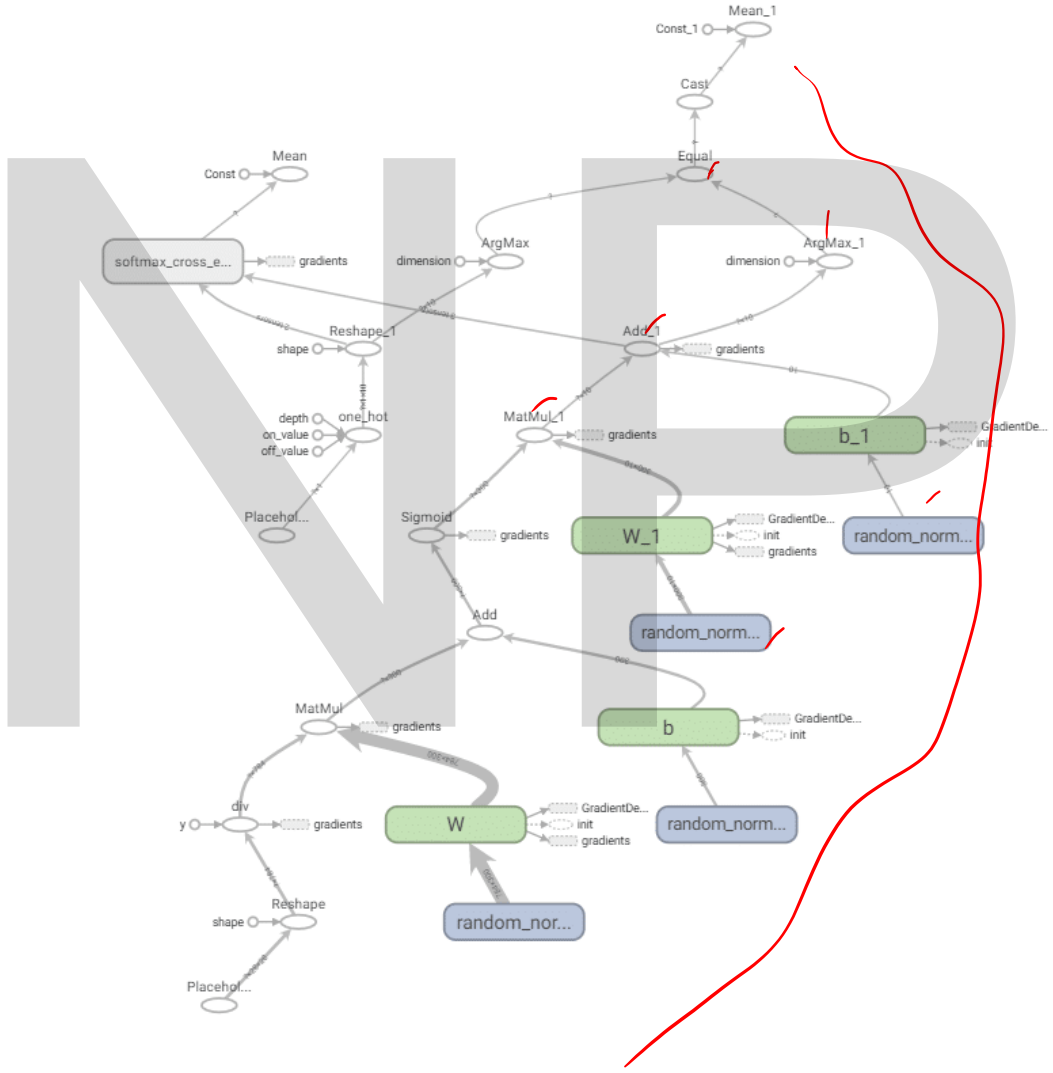
- Reinforcement Learning:
  - **[Mirhoseini et al., 2017]**
  - [Mirhoseini et al., 2018]
  - [Addanki, et al., 2019]
- ML-based cost model:
  - [Chen et al., 2018],
  - [Zhou et al., 2020],
  - [Zhang, 2020]
- Bayesian optimization:
  - [Sergeev et al., 2018],
  - [Peng et al., 2019]

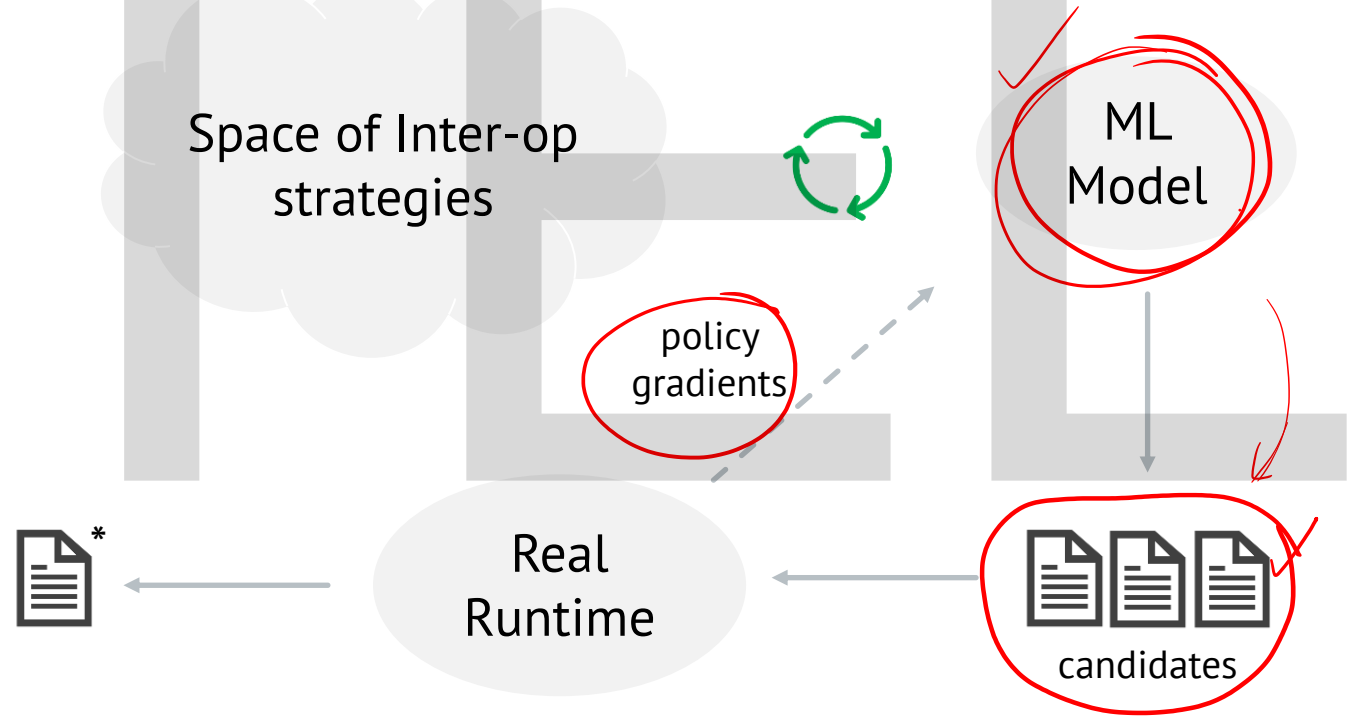
Optimization-based methods

- Dynamic programming
  - [Wang, et al., 2018]
  - [Narayanan, et al., 2019]
  - [Li, et al., 2021]
  - [Narayanan, et al., 2012]
  - [Tarnawski, et al., 2020]
  - [Tarnawski, et al., 2021]
- Integer linear programming
  - [Tarnawski, et al., 2020]
- Hierarchical optimization
  - **[Zheng, et al., 2022]**

NPTEL

ColocRL (a.k.a. Device Placement Optimization)





Mirhoseini, et al. "Device Placement Optimization with Reinforcement Learning." *ICML* 2017.

Automatic Parallelization Methods

Search-based methods

- MCMC:
  - [Jia et al., 2018]
  - [Jia et al., 2019]
- Heuristics
  - [Fan et al., 2021]

Learning-based methods

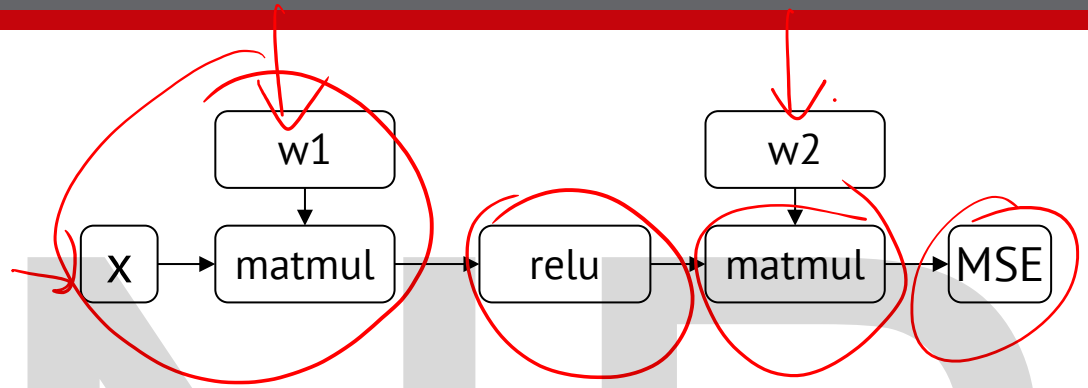
- Reinforcement Learning:
  - [Mirhoseini et al., 2017]
  - [Mirhoseini et al., 2018]
  - [Addanki, et al., 2019]
- ML-based cost model:
  - [Chen et al., 2018],
  - [Zhou et al., 2020],
  - [Zhang, 2020]
- Bayesian optimization:
  - [Sergeev et al., 2018],
  - [Peng et al., 2019]

Optimization-based methods

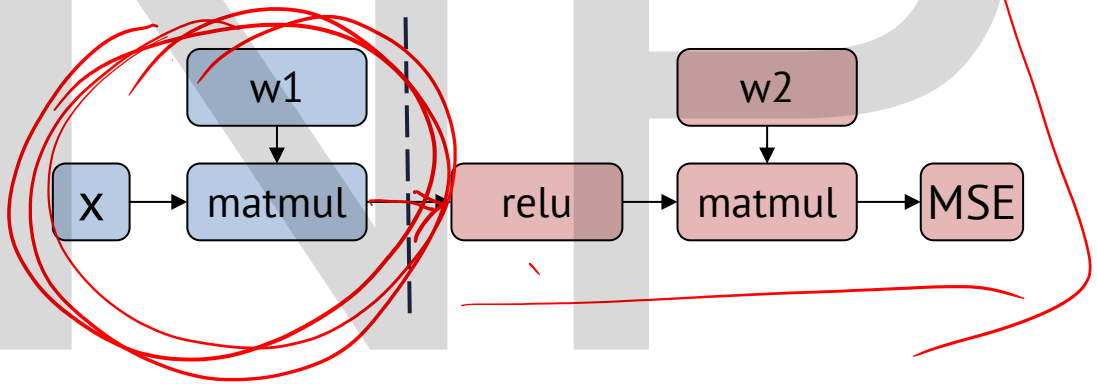
- Dynamic programming
  - [Wang, et al., 2018]
  - [Narayanan, et al., 2019]
  - [Li, et al., 2021]
  - [Narayanan, et al., 2012]
  - [Tarnawski, et al., 2020]
  - [Tarnawski, et al., 2021]
- Integer linear programming
  - [Tarnawski, et al., 2020]
- Hierarchical optimization
  - **[Zheng, et al., 2022]**

NPTEL

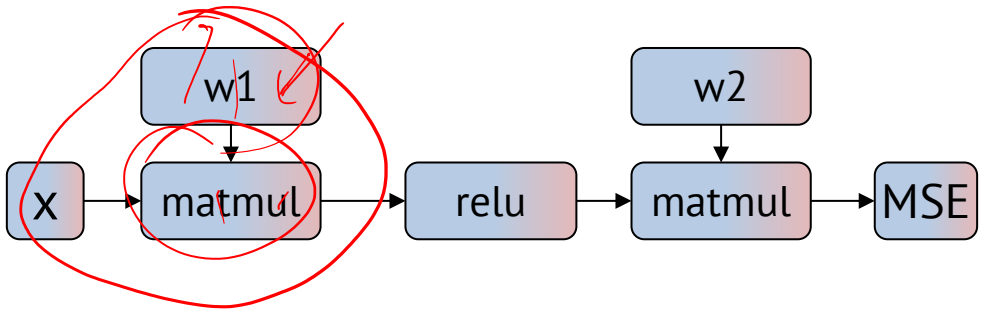
Optimization-based Method: Alpa [1]



Inter-op parallelism



Intra-op parallelism



Device 1

Device 2

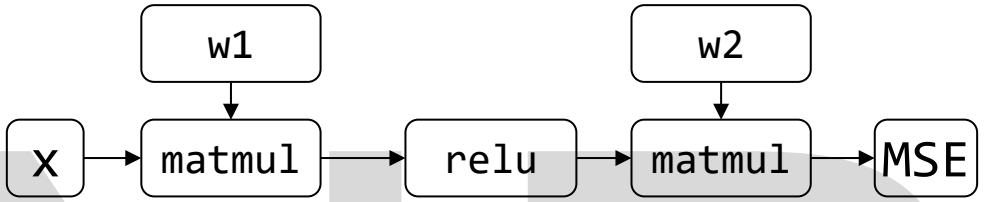
[1] Zheng, L., Li, Z., Zhang, H., Zhuang, Y., Chen, Z., Huang, Y., ... & Stoica, I. (2022). Alpa: Automating inter-and {Intra-Operator} parallelism for distributed deep learning. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)* (pp. 559-578).

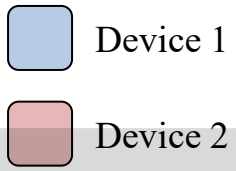
Trade-off

	Inter-operator Parallelism	**Intra-operator Parallelism**
Communication	Less	More
Device Idle Time	More	Less

NPTEL

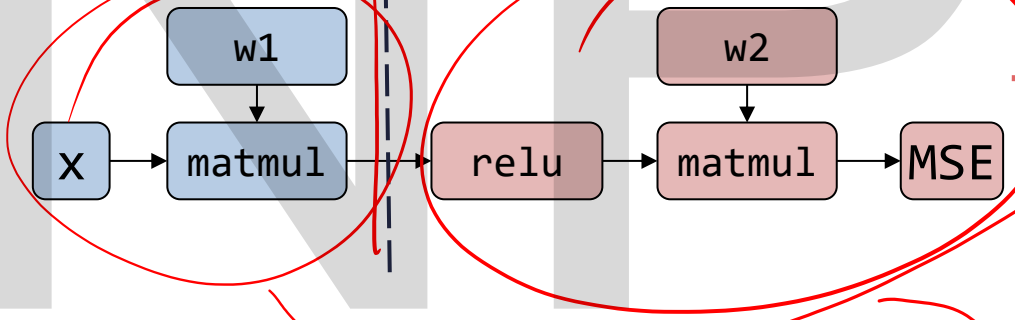
Alpa Rationale



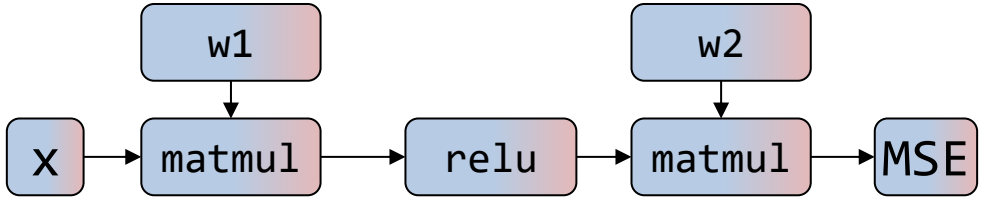


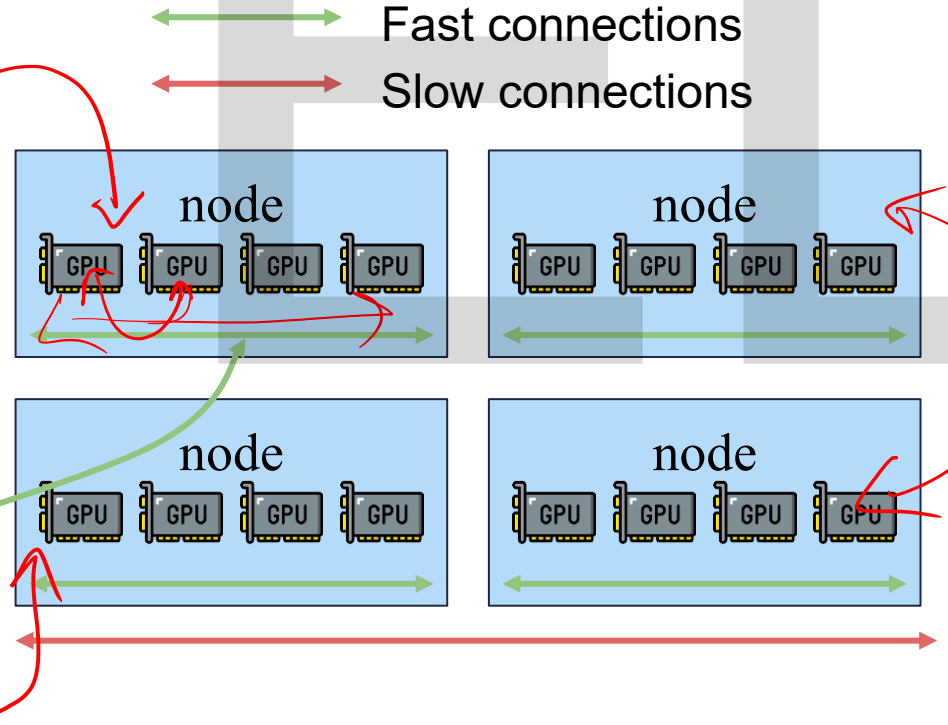
Alpa

Inter-op parallelism

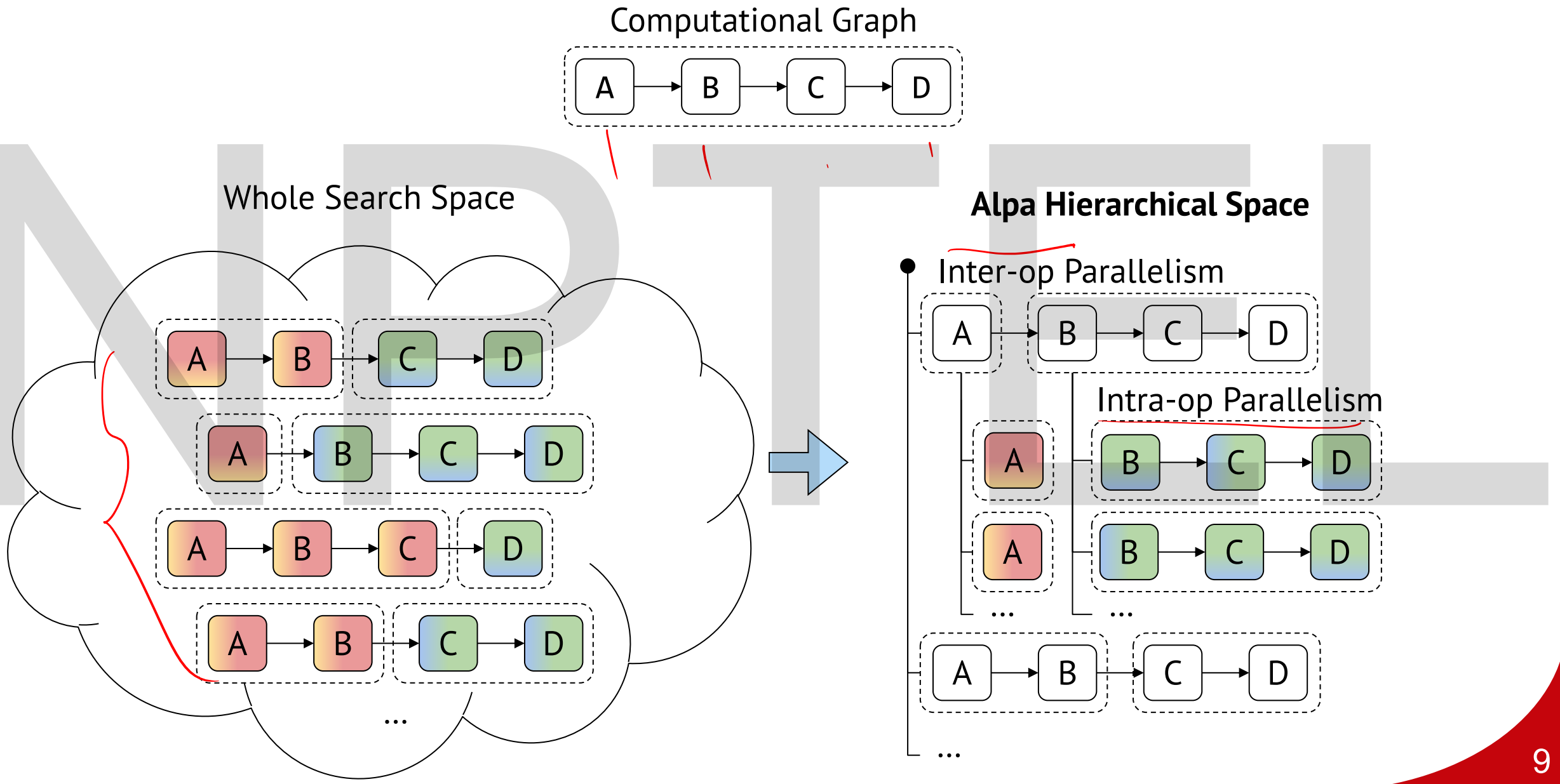


Intra-op parallelism



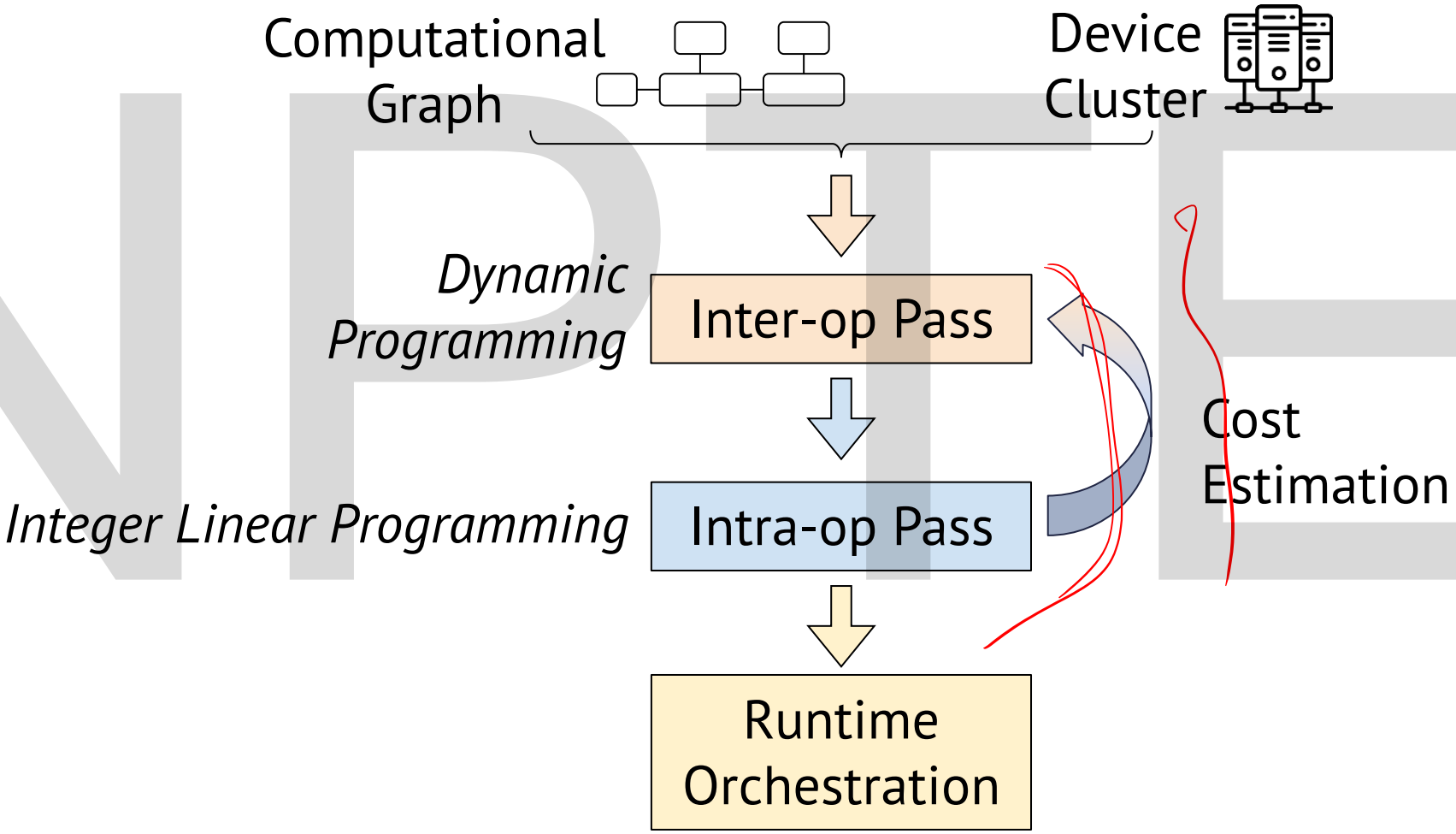


Search Space



NPTEL

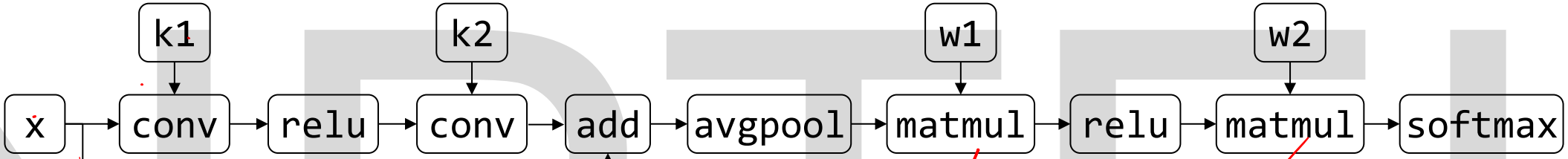
Alpa Compiler: Hierarchical Optimization



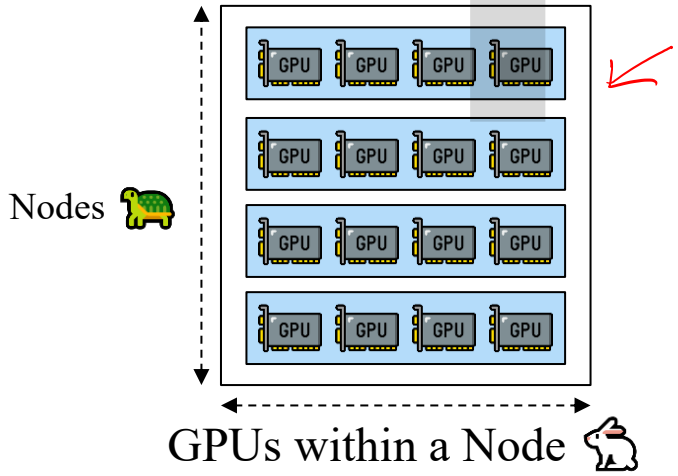
NPTEL

Inter-Op Pass

Computational Graph

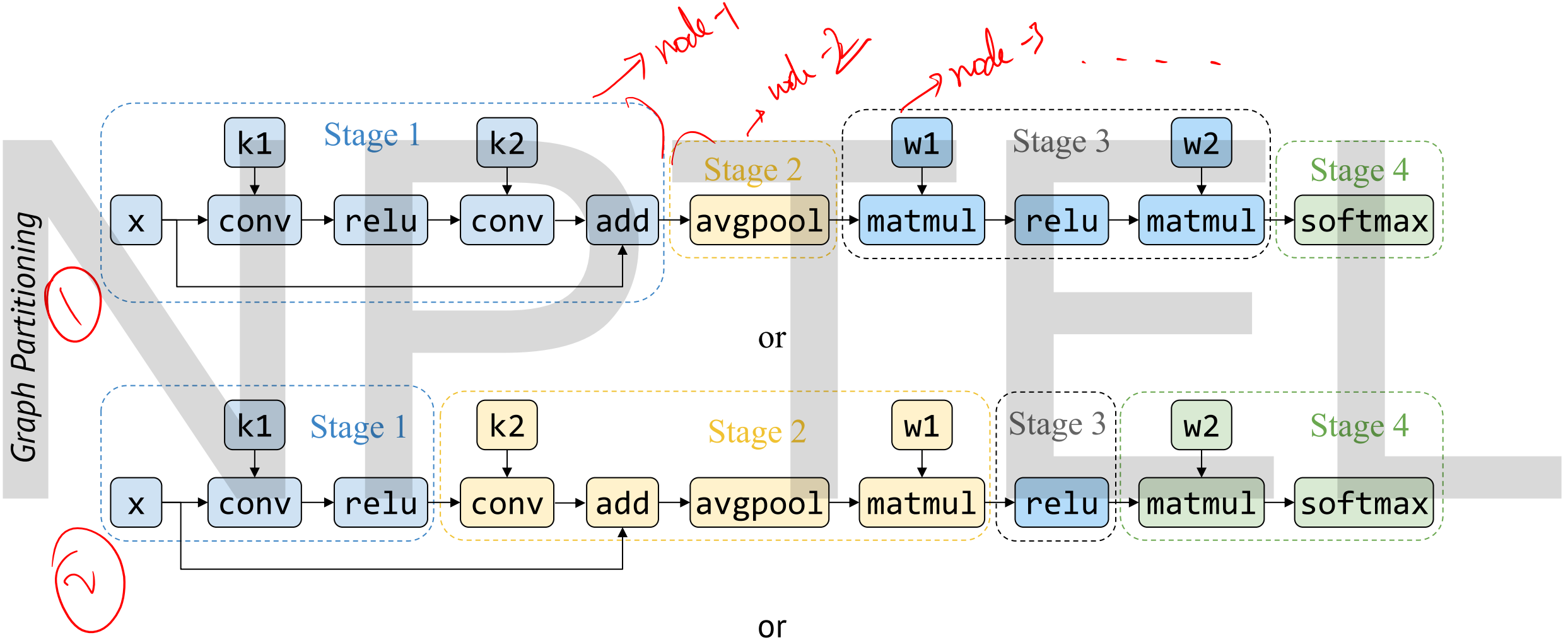


Cluster (2D Device Mesh)

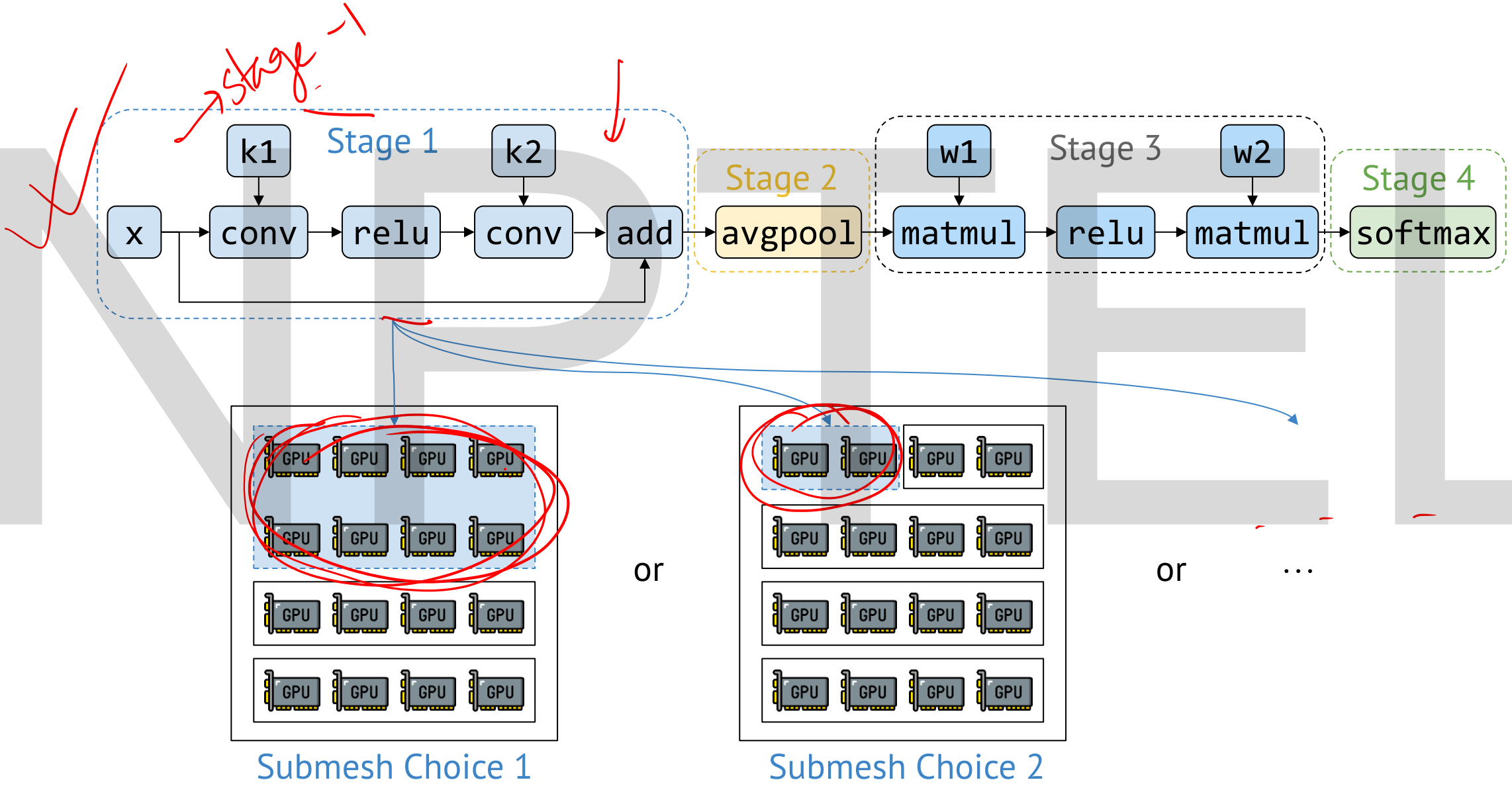


NPTEL

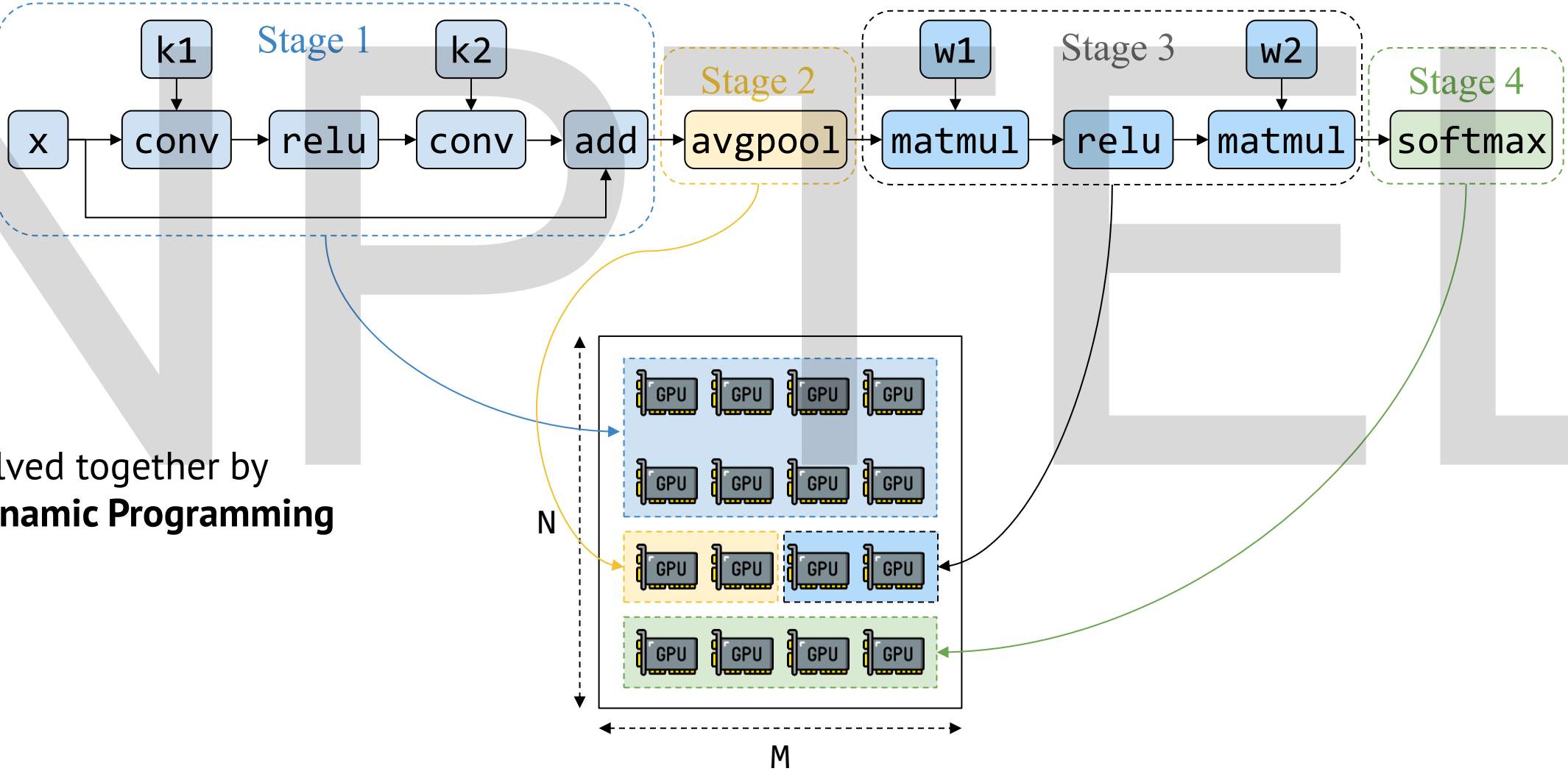
Inter-Op Pass



Inter-Op Pass

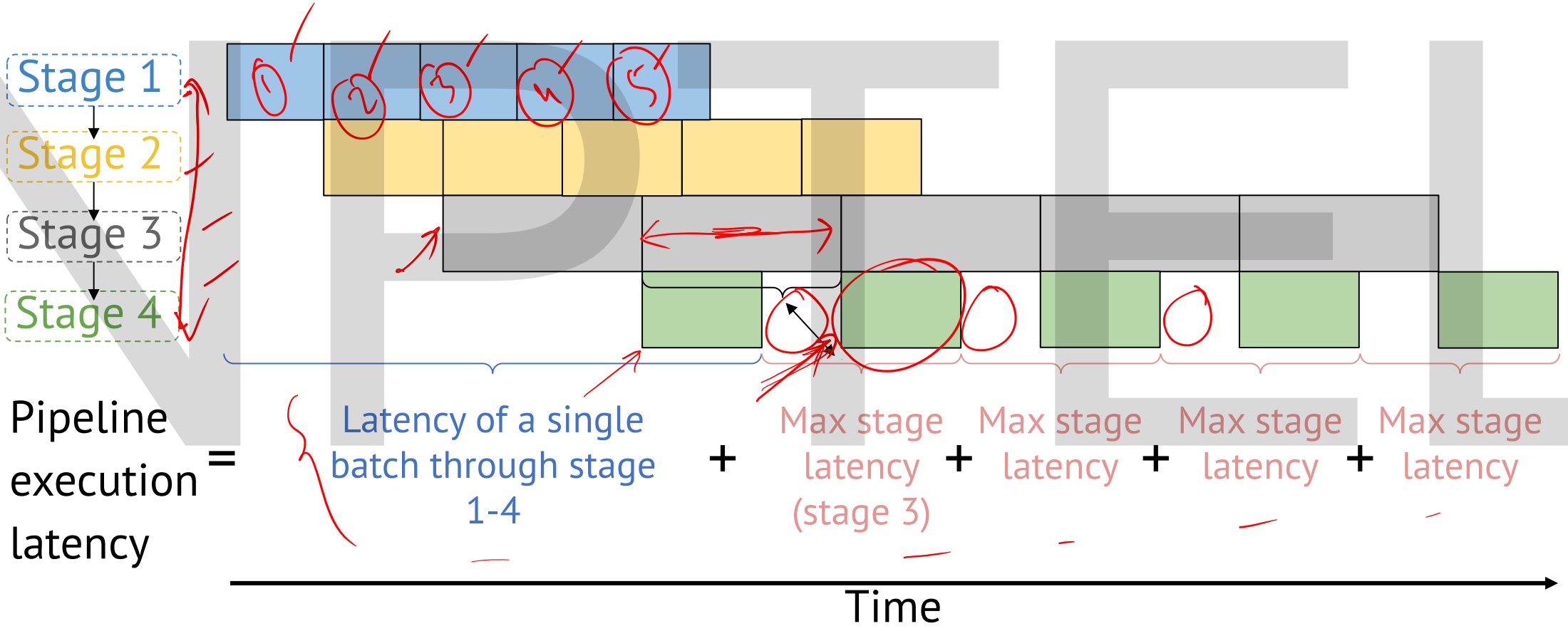


Inter-Op Pass



Solved together by
Dynamic Programming

Goal: Minimize Pipeline Execution Latency



Inter-op Pass: Dynamic Programming Formulation

$$\text{Pipeline execution latency} = \text{Latency of a single datapoint through all stages} + \text{Max stage latency} \times (\#\text{Batch} - 1)$$

Goal: Derive the best *graph partition* and *device assignment* by selecting the optimal set of (stage, submesh) pairs that minimizes this latency.

Input: Run intra-op pass and profile for all possible (stage, submesh) pairs.

Algorithm: Enumerate all possible second term and convert the first term into a 2-dimensional knapsack problem.

NPTEL



THANK YOU

Slide Courtesy: Murali Annavaram (University of Southern California), R. Govindarajan (IISc), Arkaprava Basu (IISc), Jayashree Mohan (Microsoft Research)